

Debiprasanna Sahoo

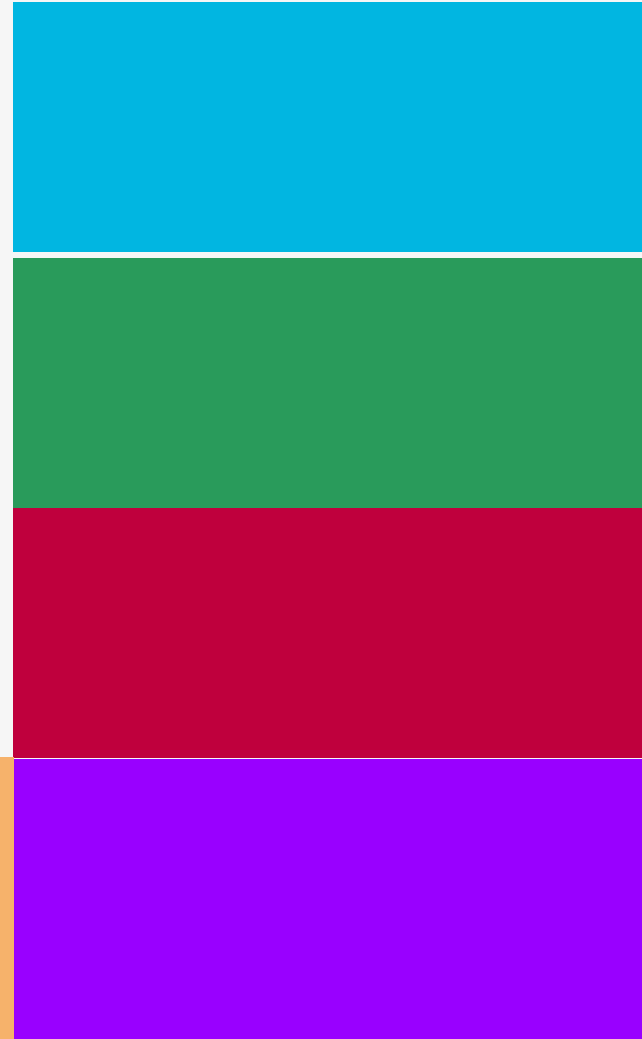
Assistant Professor

Department of Computer Science and Engineering

School of Electrical and Computer Sciences

Indian Institute of Technology Bhubaneswar

Language of the Computer: Learning via RISC-V



Reference

Computer Organization and Design: The Hardware/Software Interface- RISC-V Edition, 5th Edition, 2017

Chapter-2

David A. Patterson and John L. Henessey

Computer Systems: A Programmer's Perspective, 3rd Edition, 2016

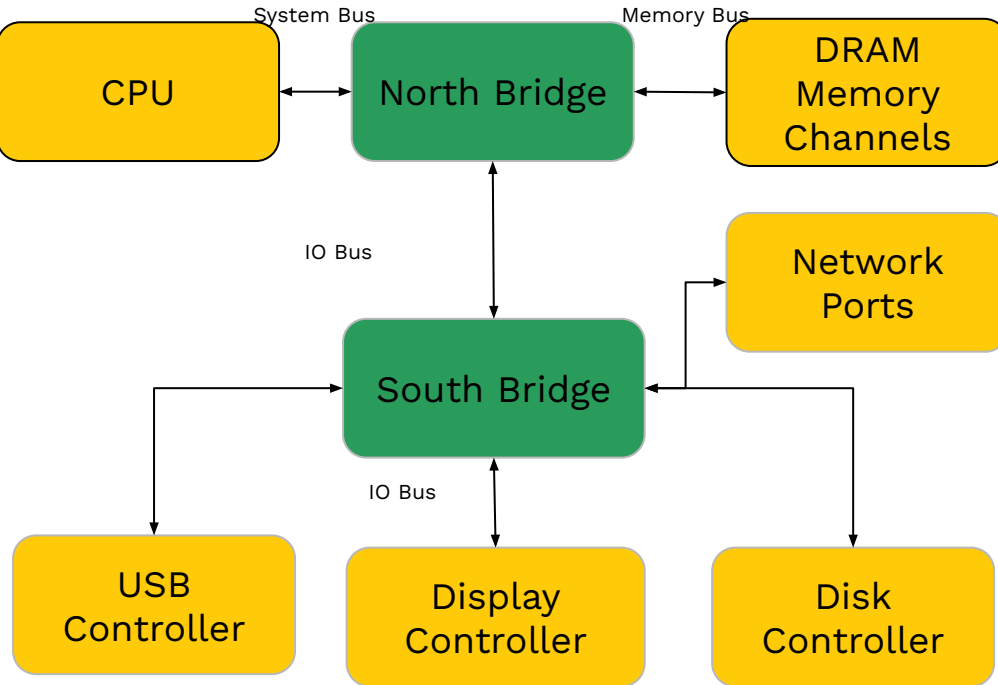
Chapter-1

Randal E. Bryant and David R. O'Hallaron

Learning Outcome: A Basic Design of RISC-V

- Visualize different logical components of a computer
- Summarize the instructions of RISC-V
- Translate a “C-Program” into assembly code
- Compare and classify the assembly instructions and translate them to machine code
- Infer how a program starts executing

A Generic Logical Organization



Computes: CPU, GPU, NPU, TPU, etc

Memory: System Memory/Main Memory

Devices: Disk, Display, Mouse, Keyboard, Network Ports

Device Controllers: Disk Controller, Display Adapter, USB Controller

Buses/Interconnects: System Bus, Memory Bus, IO Bus, Internal Buses

Bridges: North and South Bridge

What happens on “gcc -o add add.c”?

```
#include<stdio.h>
int main() {
    int a, b, c;
    a = b + c;
    printf(“%d”,a);
    return 0;
}
```

add.c

Preprocessing

```
Code related to printf
is merged
int main() {
    int a, b, c;
    a = b + c;
    printf(“%d”,a);
    return 0;
}
```

add.i

Compilation

```
main:
addi    sp,sp,-16
sw      ra,12(sp)
sw      s0,8(sp)
addi    s0,sp,16
lw      a4,-20(s0)
lw      a5,-24(s0)
add     a5,a4,a5
sw      a5,-36(s0)
addi    a0,a0,zero
j       printf
lw      ra,12(sp)
lw      s0,8(sp)
addi    sp,sp,16
jr      ra
```

add.s

Assembler

```
1000000 01111 00010 000 00010 010011
0000000 01100 00001 000 00010 100011
0000000 01000 01000 000 00010 100011
0000000 01000 01000 000 00010 010011
1111111 01100 01110 000 01010 000011
1111111 01000 01111 000 01010 000011
0000000 00101 00100 000 00101 110011
1111110 11100 00101 000 01000 000011
0000000 01010 01010 000 00000 010011
Binary for Jump related to printf
0000000 01100 00001 000 01010 000011
0000000 01000 01000 000 01010 000011
0000000 01000 00010 000 00010 010011
0000000 00000 00000 000 00001 111111
```

add.o

Linking

Binary related to
library

printf.o

Linking

```
Binary related to library
1000000 01111 00010 000 00010 010011
0000000 01100 00001 000 00010 100011
0000000 01000 01000 000 00010 100011
0000000 01000 01000 000 00010 010011
1111111 01100 01110 000 01010 000011
1111111 01000 01111 000 01010 000011
0000000 00101 00100 000 00101 110011
1111110 11100 00101 000 01000 000011
0000000 01010 01010 000 00000 010011
Binary for Jump related to printf
0000000 01100 00001 000 01010 000011
0000000 01000 01000 000 01010 000011
0000000 01000 00010 000 00010 010011
0000000 00000 00000 000 00001 111111
```

add

Command The Computer

Compiler

The compiler turns the source language program into an assembly program

```
Code related to printf
is merged
int main() {
    int a, b, c;
    a = b + c;
    printf("%d",a);
    return 0;
}
```

add.i

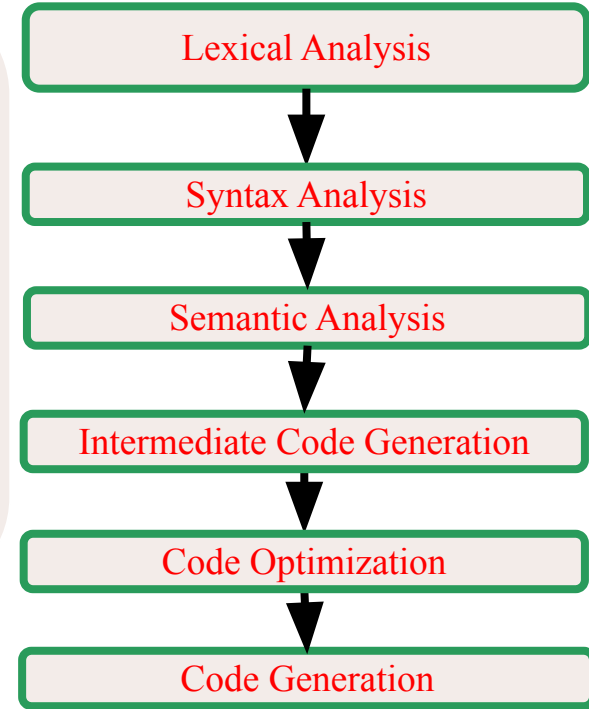
Compilation

```
main:
addi    sp,sp,-16
sw      ra,12(sp)
sw      s0,8(sp)
addi    s0,sp,16
lw      a4,-20(s0)
lw      a5,-24(s0)
add     a5,a4,a5
sw      a5,-36(s0)
addi    a0,a0,zero
j       printf
lw      ra,12(sp)
lw      s0,8(sp)
addi    sp,sp,16
jr      ra
```

add.s

How Compilation Happens?

- **Lexical Analysis:** Parses the program into tokens by scanning the file
- **Syntax Analysis:** Validates against the grammar of language to build the parse tree
- **Semantic Analysis:** Checks the meaning of the code, type management, other errors related to syntax
- **Intermediate Code Generation:** Generates assembly code. This phase generates an intermediate code that can be transformed into machine code.
- **Code Optimization:** Removes dead code, reorder instructions and many more stuff
- **Code Generation:** Generate final assembly code after optimization



Instruction and Instruction Set

Instruction: The commands understood by a given architecture

Instruction Set: The vocabulary of commands understood by a given architecture

The Core Execution: CPU/GPU/NPU/Accelerator

Instruction handling unit	Data handling unit (load store unit)	Register Set	Execution Unit	Control Unit
• Reading instruction • Decoding instruction	• Reading memory • Writing memory	• Reading registers • Writing registers	• Arithmetic and logic unit • Floating point unit • Integer unit	• Hardwired Controlled • Micro-programmed Controlled

The Memory

Main Memory

- Typically, embedded boards ~ 2/4 GB, Mobile ~ 4/8/16 GB, Laptops/Desktops ~ 8/16/32 GB, Workstations ~ 64/128 GB, Servers ~ 128GB/2TB
- OS-controlled allocation
- Hardware-controlled access

Secondary Storage

- TBs now for laptops, desktops, workstations, servers
- OS-controlled allocation/access

Cache Memory (8KB to 64 MB)

- Hardware-controlled allocation/access

General Purpose Registers

Where are GPRs presents? *Storage is internal to the processor and is faster than memory*

Who allocates Registers? *Registers are more efficiently allocated by compilers*

How many registers are sufficient? *Depends on the smartness of the compiler.*

Control and Status Registers

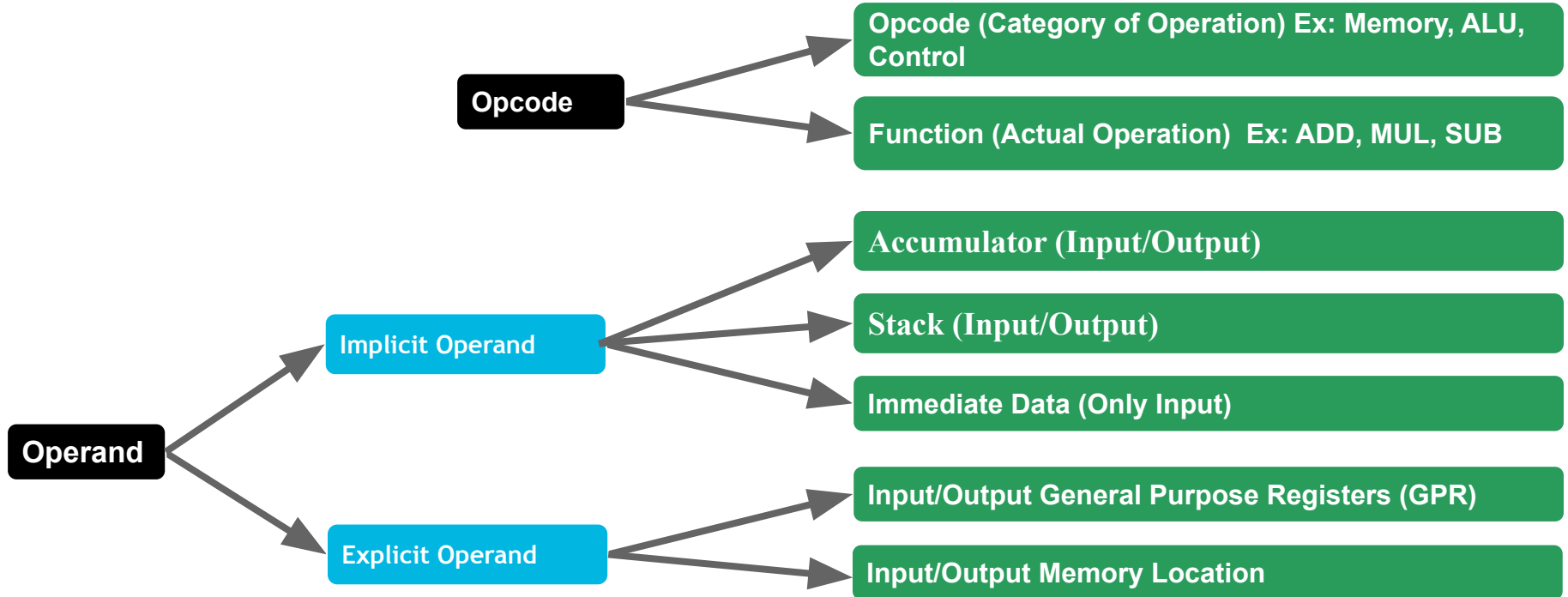
What are CSRs?

*Control registers manage system level functions like memory management and process/thread scheduling
Status registers like flag and interrupt registers are used to process dependent instructions,*

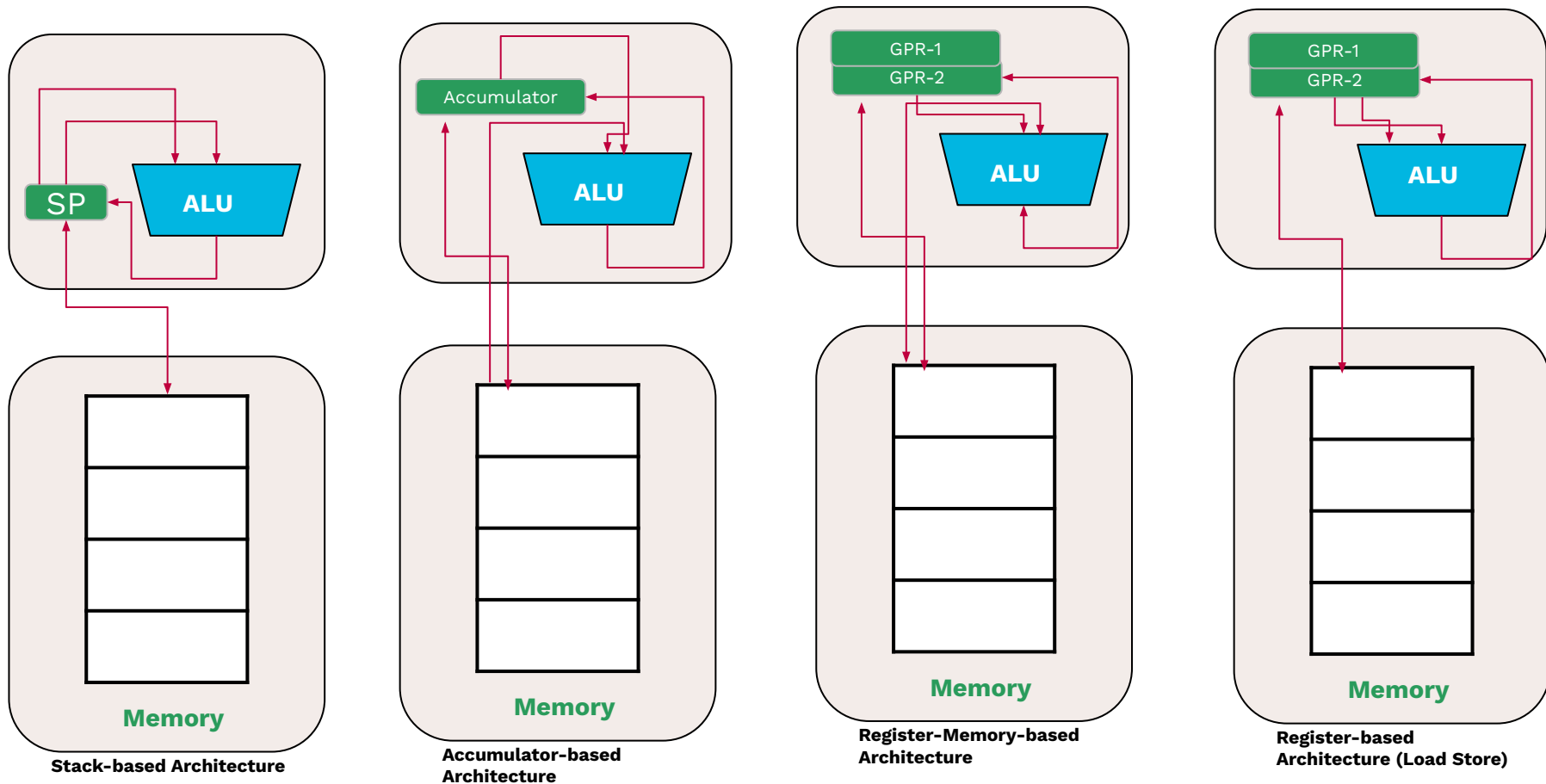
Who interacts with Control Registers?

OS and other system softwares sets them.

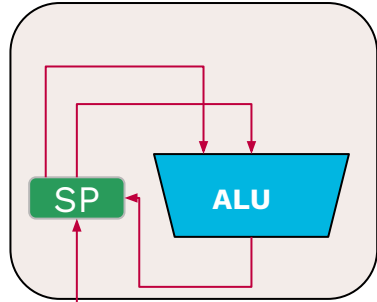
Components of Instruction



Classes of Instructions



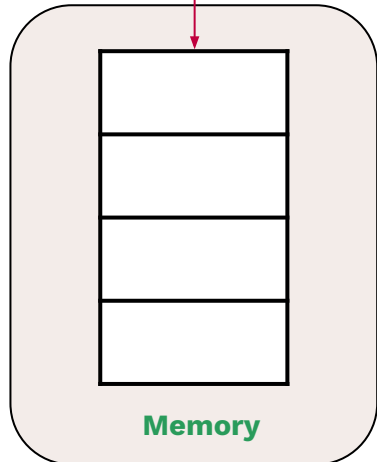
Stack-based Instructions



```
int a, b, c;  
a = b + c;
```



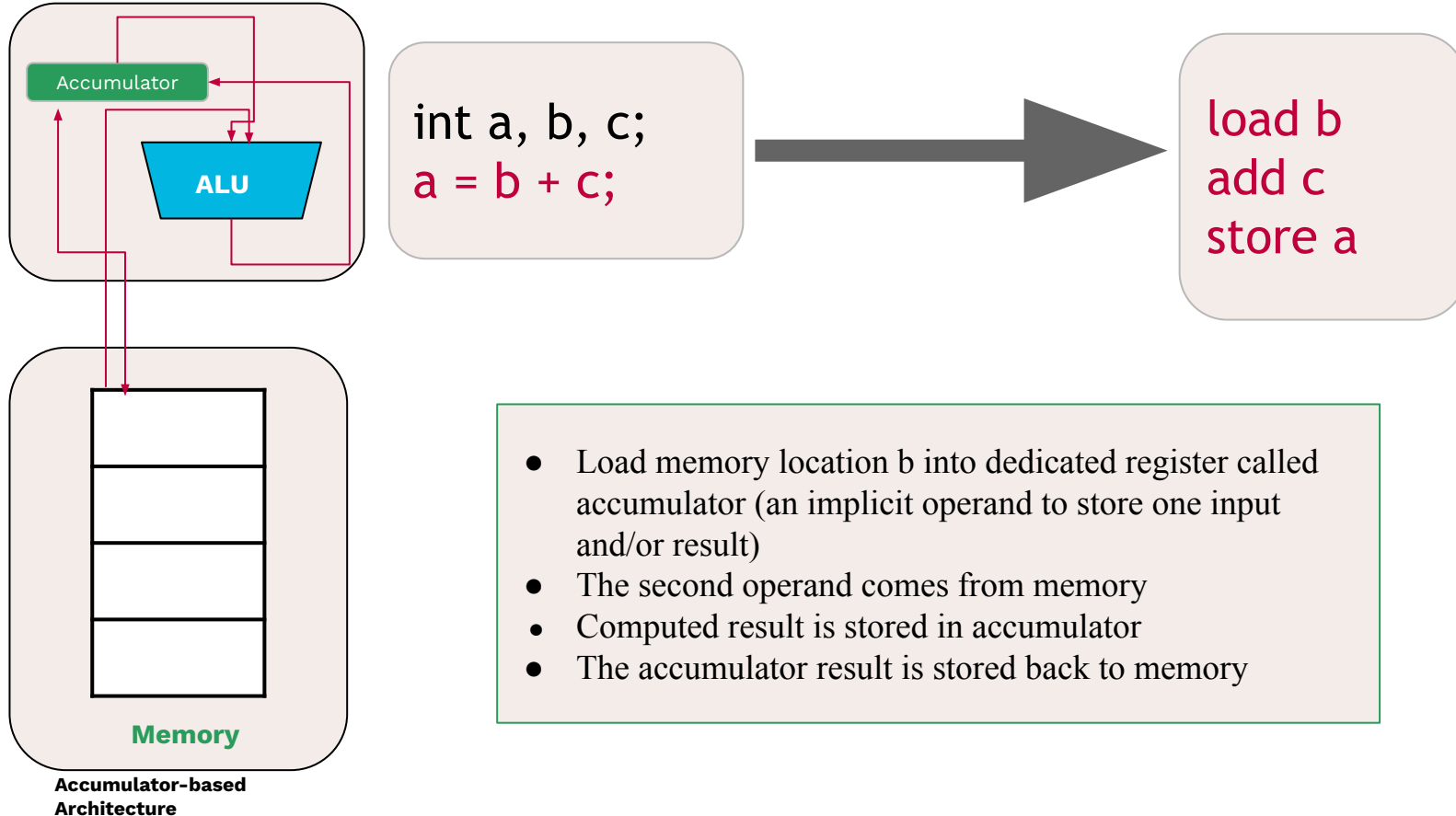
```
push b  
push c  
add  
pop a
```



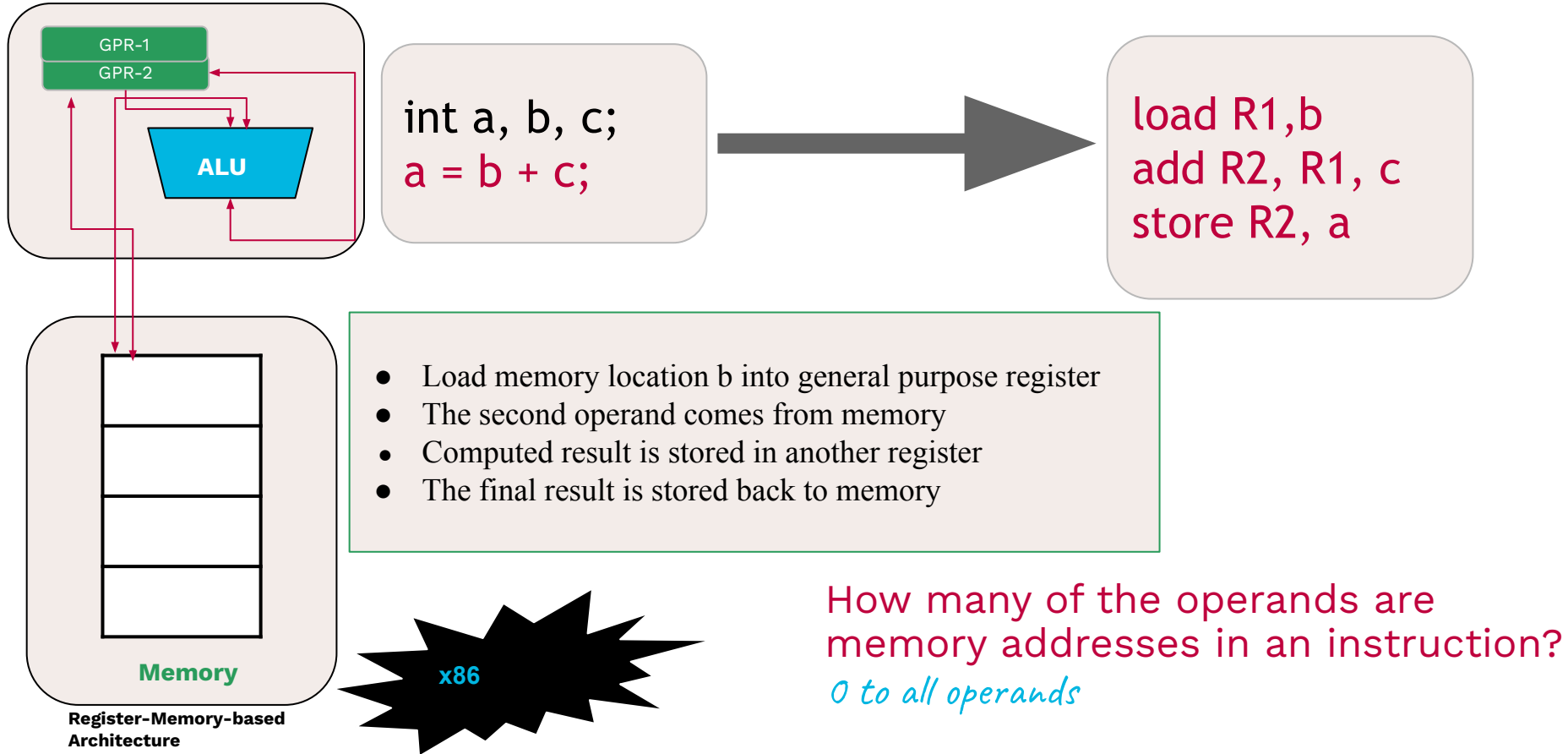
Stack-based Architecture

- Push b and c to the stack memory
- Add automatically pops the two top of the stack operates on it and pushes the result back into the stack
- Finally, we pop the result from the stack memory and store into the location

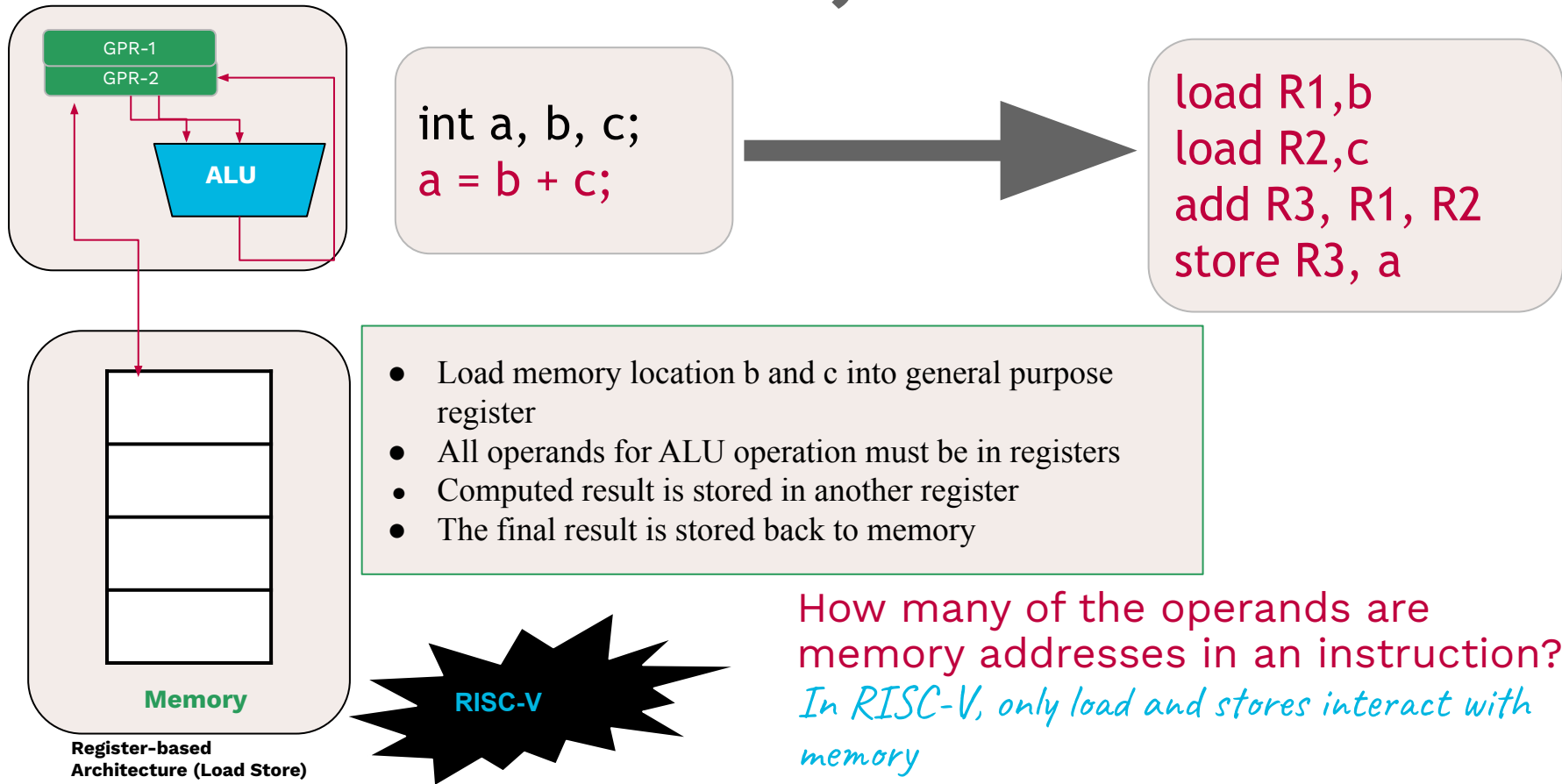
Accumulator-based Instructions



Register-Memory Instructions



Register-Register Instructions (or Load-Store Architecture)



RISC-V INSTRUCTION SET ARCHITECTURE

RISC-V Register Set

RegNo#	ABI	Comment	RegNo#	ABI	Comment
X0	Zero	Hardwired Zero	X16	a6	Function Argument 6
X1	ra	Return Address	X17	a7	Function Argument 7
X2	sp	Stack Pointer	X18	s2	Saved register 2
X3	gp	Global Pointer	X19	s3	Saved register 3
X4	tp	Thread Pointer	X20	s4	Saved register 4
X5	t0	Temporary 0	X21	s5	Saved register 5
X6	t1	Temporary 1	X22	s6	Saved register 6
X7	t2	Temporary 2	X23	s7	Saved register 7
X8	s0/fp	Saved register 0/ Frame Pointer	X24	s8	Saved register 8
X9	s1	Saved register 1	X25	s9	Saved register 9
X10	a0	Function Argument 0/Return Value 0	X26	s10	Saved register 10
X11	a1	Function Argument 1/Return Value 1	X27	s11	Saved register 11
X12	a2	Function Argument 2	X28	t3	Temporary 3
X13	a3	Function Argument 3	X29	t4	Temporary 4
X14	a4	Function Argument 4	X30	t5	Temporary 5
X15	a5	Function Argument 5	X31	t6	Temporary 6

RISC-V Assembly Instruction Set: Arithmetic and Logic (Nos: 9)

Category	Instruction	Example	Meaning	Comment
Arithmetic	Add	ADD X5, X6, X7	$X5 = X6 + X7$	Add two registers
	Substract	SUB X5, X6, X7	$X5 = X6 - X7$	subtract 2nd from 1st registers
	Add Immediate	ADDI X5, X6, 10	$X5 = X6 + 10$	Add immediate to a register
Logical	AND	AND X5, X6, X7	$X5 = X6 \& X7$	Bitwise AND two registers
	OR	OR X5, X6, X7	$X5 = X6 X7$	Bitwise OR two registers
	Exclusive OR	XOR X5, X6, X7	$X5 = X6 \wedge X7$	Bitwise XOR two registers
	And Immediate	ANDI X5, X6, 10	$X5 = X6 \& 10$	Bitwise AND register to immediate
	Or Immediate	ORI X5, X6, 10	$X5 = X6 10$	Bitwise OR register to immediate
	Exclusive Or Immediate	XORI X5,X6, 10	$X5 = X6 \wedge 10$	Bitwise XOR register to immediate

C vs Assembly Code: Arithmetic and Logic Instruction

```
int a, b, c, d, e;  
a = b + c;  
f = d + e;  
a = a - f;
```

a ~ X5, b ~ X6, c ~ X7



f ~ X28, d ~ X29, e ~
X30

```
add X5, X6, X7  
add X28, X29, X30  
sub X5, X5, X28
```

RISC-V Assembly Instruction Set: Memory Operations (Nos: 9)

Category	Instruction	Example	Meaning	Comment
Data Transfer	Load Word	LW X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 32 bits from $X6 + 10$. use as signed
	Load Word Unsigned	LWU X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 32 bits from $X6 + 10$. use as unsigned
	Load Half Word	LH X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 16 bits from $X6 + 10$. use as signed
	Load Half Word Unsigned	LHU X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 16 bits from $X6 + 10$. use as unsigned
	Load Byte	LB X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 8 bits from $X6 + 10$. use as signed
	Load Byte Unsigned	LBU X5, 10(x6)	$X5 = \text{Memory}[10 + X6]$	Read 8 bits from $X6 + 10$. use as unsigned
	Store Word	SW X5, 10(x6)	$\text{Memory}[10 + X6] = X5$	Write 32 bits to $X6 + 10$
	Store Half Word	SH X5, 10(x6)	$\text{Memory}[10 + X6] = X5$	Write 16 bits to $X6 + 10$
	Store Byte	SB X5, 10(x6)	$\text{Memory}[10 + X6] = X5$	Write 8 bits to $X6 + 10$

C vs Assembly Code: Memory Instruction

```
int a, b, c, d, e;
```

```
a = b + c;
```

```
f = d + e;
```

```
a = a - f;
```



```
lw    t0,-20(s0)
```

```
lw    t1,-24(s0)
```

```
add    t0,t0,t1
```

```
lw    t2,-28(s0)
```

```
lw    t3,-32(s0)
```

```
add    t2,t2,t3
```

```
sub    t0,t0,t2
```

```
sw    t0,-36(s0)
```

RISC-V Assembly Instruction Set:

Conditional Jump (Nos: 6)

Category	Instruction	Example	Meaning	Comment
Conditional Jump	Branch if Equal	BEQ X5, X6, 10	If $X5 == x6$ then $PC = PC + 10$	Compare equality
	Branch if not equal	BNE X5, X6, 10	If $X5 != x6$ then $PC = PC + 10$	Compare non-equality
	Branch if less than	BLT X5, X6, 10	If $X5 < x6$ then $PC = PC + 10$	Branch when less
	Branch if less unsigned	BLTU X5, X6, 10	If $X5 < x6$ then $PC = PC + 10$	Branch when less unsigned
	Branch if greater or equal than	BGE X5, X6, 10	If $X5 \geq x6$ then $PC = PC + 10$	Branch when greater
	Branch if greater or equal than unsigned	BGEU X5, X6, 10	If $X5 \geq x6$ then $PC = PC + 10$	Branch when greater unsigned

C vs Assembly Code: Control Structure

```
int a, b, c, d, e;  
if(b < c){  
    a = b + c;  
}  
else{  
    a = d + e;  
}
```



```
lw    a4, -20(s0)  
lw    a5, -24(s0)  
bge   a4, a5, .L2  
add   a5, a4, a5  
sw    a5, -36(s0)  
j     .L4  
.L2:  
lw    a4, -28(s0)  
lw    a5, -32(s0)  
add   a5, a4, a5  
sw    a5, -36(s0)  
.L4:
```

RISC-V Assembly Instruction Set: Shift and Unconditional Jump (Nos: 8)

Category	Instruction	Example	Meaning	Comment
Shift	Shift Left Logical	SLL X5, X6, X7	$X5 = X6 \ll X7$	Shift left by value in X7
	Shift Right Logical	SRL X5, X6, X7	$X5 = X6 \gg X7$	Shift right by value in X7
	Shift Right Arithmetic	SRA X5, X6, X7	$X5 = X6 \ggg X7$	Shift right arithmetic by value in X7
	Shift Left Logical Immediate	SLLI X5, X6, 10	$X5 = X6 \ll 10$	Shift left by immediate value
	Shift Right Logical Immediate	SRLI X5, X6, 10	$X5 = X6 \gg 10$	Shift right by immediate value
	Shift Right Arithmetic Immediate	SRAI X5, X6, 10	$X5 = X6 \ggg 10$	Shift right arithmetic immediate value
Unconditional Jump	Jump and Link	JAL X1, 100	$X1 = PC + 4$ $PC = PC + 100$	Call
	Jump and Link Register	JALR X1, 100(X5)	$X1 = PC + 4$ $PC = X5 + 100$	Return is equivalent to JALR X0, 0(X1). Used for pointer to function also.

C vs Assembly Code: Function Call and Return

```
void function() {  
    int a, b, c, d, e;  
    if(b < c) {  
        a = b + c;  
    }  
    else {  
        a = d + e;  
    }  
}  
  
int main() {  
    function();  
    return 0;  
}
```

function:

```
addi    sp,sp,-48  
sw      ra,44(sp)  
sw      s0,40(sp)  
addi    s0,sp,48  
lw      a4,-20(s0)  
lw      a5,-24(s0)  
bge     a4,a5,.L2  
add     a5,a4,a5  
sw      a5,-36(s0)  
j       .L4
```

.L2:

```
lw      a4,-28(s0)  
lw      a5,-32(s0)  
add     a5,a4,a5  
sw      a5,-36(s0)
```

.L4:

```
lw      ra,44(sp)  
lw      s0,40(sp)  
addi    sp,sp,48  
jr      ra
```

main:

```
addi    sp,sp,-16  
sw      ra,12(sp)  
sw      s0,8(sp)  
addi    s0,sp,16  
j       function  
addi    a0,a0,zero  
lw      ra,12(sp)  
lw      s0,8(sp)  
addi    sp,sp,16  
jr      ra
```

RISC-V Assembly Instruction Set: Set-based on Compare and Long Immediate (Nos. 6)

Category	Instruction	Example	Meaning	Comment
Set based on Compare	Set Less than	SLT X5, X6, X7	$X5 = X6 < X7 ? 1 : 0$	Set when one register less than other
	Set Less than unsigned	SLTU X5, X6, X7	$X5 = X6 < X7 ? 1 : 0$	Set when one register less than other
	Set less than immediate	SLTI X5, X6, 10	$X5 = X6 < 10 ? 1 : 0$	Set when one register less than immediate
	Set less than immediate unsigned	SLTIU X5, X6, 10	$X5 = X6 < 10 ? 1 : 0$	Set when one register less than immediate
Long Immediate	Long Unsigned Immediate	LUI X5, 10	$X5 = 10 \ll 12$	Prepare a long immediate will last 12 bits reset
	Load Word Unsigned	AUIPC X5, 10	$X5 = PC + 10 \ll 12$	Prepare a long immediate with last 12 bits reset and added to program counter

RISC-V Assembly Instruction Set: Misc

Category	Instruction	Example	Meaning	Comment
No Operation	No Operation	NOP	Do nothing	Do Nothing
System	System Call	ECALL	Make system call	Jump and enter privilege functions
	Break	EBREAK	Introduce break in debugging	-

**Make The Computer
Understand Commands**

Assemblers

The assembler turns the assembly language program into an object file, which is a combination of:

- machine language instructions
- data
- information needed to place instructions and data properly in memory.

```
main:
addi  sp,sp,-16
sw    ra,12(sp)
sw    s0,8(sp)
addi  s0,sp,16
lw    a4,-20(s0)
lw    a5,-24(s0)
add   a5,a4,a5
sw    a5,-36(s0)
addi  a0,a0,zero
j      printf
lw    ra,12(sp)
lw    s0,8(sp)
addi  sp,sp,16
jr    ra
```

add.s

Assembler

```
1000000 01111 00010 000 00010 010011
0000000 01100 00001 000 00010 100011
0000000 01000 01000 000 00010 100011
0000000 01000 01000 000 00010 010011
1111111 01100 01110 000 01010 000011
1111111 01000 01111 000 01010 000011
0000000 00101 00100 000 00101 110011
1111110 11100 00101 000 01000 000011
0000000 01010 01010 000 00000 010011
Binary for Jump related to printf
0000000 01100 00001 000 01010 000011
0000000 01000 01000 000 01010 000011
0000000 01000 00010 000 00010 010011
0000000 00000 00000 000 00001 111111
```

add.o

Header: Specifies size and position

Code in form of machine language

Static and Extern Data

Relocation information: Move code anywhere

Symbol Table

Debugging Information: Did you use gdb?

Object File

- Assemblers keep track of labels used in branches and data transfer instructions in a symbol table.
- Symbol table matches names of labels to the addresses of the memory words that instructions occupy

Addressing Mode

Definition: One of several addressing regimes delimited by their varied use of operands and/or addresses.

Meaning: The way in which the operand of an instruction or code is specified in machine language

RISC-V Instruction Set and Addressing Modes

Register Type (R-Type)

Upper Immediate (U-Type)

Immediate Type (I-Type)

Branch Type (B-Type)

Store Type (S-Type)

Jump Type (J-Type)

Register Type (R-Type) Instruction Format

func7[31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
0000000	src2	src1	ADD/SLT/SLTU/AND /OR/XOR/SLL/SRL	dest	OP = 51
0100000	src2	src1	SUB/SRA	dest	OP = 51

- Operands are attempted to be placed in same location for easy decoding
- funct3 determines the actual operation to be executed
- funct7 are the extension bits for existing and new extensions
- 30th bit is used to distinguish SUB and Arithmetic Right Shift from other ALU operations

ADD X5, X6, X7 => 0000000 00110 00111 000 00101 0110011

SUB X28, X29, X30 => 0100000 11101 11110 000 11100 0110011

Immediate Type (I-Type): Arithmetic Instructions

immediate[31-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
immediate[11:0]	src	ADDI/SLTI[U]/ ANDI/ORI/XORI	dest	OP-IMM = 19

- **Immediate**: One of the operand is a constant within the instruction itself. Other operands are present in registers.
- func3 bits selects between the 6 immediate ALU operations: Add, And, Or, Xor, set less than immediate, set less than immediate unsigned
- I-Type instruction is derived from and is similar to R-Type Instruction to have an easy hardware design.

R-Type	func7[31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate[31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]

ADDI X5, X6, 10 => 000000001010 00110 000 00101 0010011

ORI X28, X29, 30 => 000000011110 11101 101 11100 0010011

NOP Instruction

immediate[31-20]	rs1 [19-15]	func3 [14-12]	rd[11-7]	opcode [6-0]
0	0	ADDI	0	OP-IMM

- The NOP instruction does not change any user-visible state, except for advancing the pc.
- NOP is encoded as ADDI x0, x0, 0.
- Remember! X0 is set to 0
- Encoding is same as I-Type Instruction

NOP => 000000 00000 000 00000 0010011

Immediate Type (I-Type): Shift Instructions

func7 [31-25]	Immediate [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
0000000	shamt[4:0]	src	SLLI=001/SRLI=101	dest	OP-IMM = 19
0100000	shamt[4:0]	src	SRAI=101	dest	OP-IMM = 19

- func3 bits selects rest 2 ALU operations: Shift Left, Shift Right
- shamt is the short form for shift amount. Why only 5 bits?
- All func3 bits are exhausted, so bit 30 will be set to have extensions of I-Type instructions

R-Type	func7 [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate[31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	immediate[24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]

SRLI X5, X6, 10 => 0000000 01010 00110 001 00101 0010011

SRAI X28, X29, 30 => 0100000 11110 11101 000 11100 0010011

I-Type: Load Instructions

immediate[31-20]	rs1 [19-15]	func3 [14-12]	rd[11-7]	opcode [6-0]
offset[11-0]	src	width(LW/LD/LH/ LB/LWU/LHU/LBU)	dest	LOAD=3

- func3 bits selects between 7 Load operations: Load Word, Load Double Word, Load Half Word, Load Byte, Load Word Unsigned, Load Half Word Unsigned, Load Byte Unsigned
- The operand is at the memory location whose address is the sum of a register and a constant in the instruction.
- Uses displacement addressing mode
- Uses an encoding similar to normal I-Type instructions.

R-Type	func7[31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate[31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	immediate[24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]

LW X5, 10(X6) => 000000001010 00110 000 00101 0000011

LWU X28, 30(X29) => 000000011110 11101 100 11100 0000011

Store Type (S-Type)

Immediate [31-25]	rs2[24-20]	rs1 [19-15]	func3 [14-12]	Immediate [11-7]	opcode[6-0]
offset[11:5]	data	src	width (SD/SW/SHW/SB)	offset[4:0]	STORE=35

- func3 bits selects between 3 Store Operations: Store Double, Store Word, Store Half Word, Store Byte
- Uses displacement addressing mode

R-Type	func7 [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	immediate[24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
S-Type	immediate [31-25]	rs2[24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]

SW X5, 10(X6) => 00000000 00101 00110 000 01010 0100011

SB X28, 30(X29) => 00000000 11100 11101 100 11110 0100011

Conditional Branch Type (B-Type)

immediate[31-25]	rs2[24-20]	rs1 [19-15]	func3 [14-12]	immediate[11-7]	opcode [6-0]
offset(12,10-5)	src2	src1	BEQ/BNE/BLT[U]/BGE[U]	offset(11,4-1)	BRANCH=99

- func3 bits selects between 6 branch operations: branch equal, branch not equal, branch less than, branch less than unsigned, branch greater equal, branch greater equal unsigned
- The branch address is the sum of the PC and a constant in the instruction.
- Uses PC relative addressing mode.
- Encoding is similar to S-Type instructions.

R-Type	func7 [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	immediate[24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
S-Type	immediate [31-25]	rs2[24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]
B-Type	immediate[31-25]	rs2[24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]

Unconditional Branch: Jump Type (J-Type)

immediate [31:12]				rd [11-7]	opcode [6-0]
offset(20)	offset(10:1)	offset(11)	offset(19-12)	dest	JAL=111
immediate [31-20]	rs1 [19-15]	func3 [14-12]		rd [11-7]	opcode [6-0]
offset[11-0]	base	0		dest	JALR=103

- The offset/imm is sign-extended and added to the pc to form the jump target address.
- JAL stores the address of the next instruction into register rd, i.e., $rd = pc + 4$
- Used in switch case and procedure call or return
- With JAL, $pc = pc + imm20$
- With JALR, $pc = rs + imm12$ (JALR is an I-Type Instruction)

R-Type	func7 [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	immediate[24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
S-Type	immediate [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]
B-Type	Immediate [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]
J-Type (JAL)	immediate [31:12]				rd [11-7]	opcode [6-0]
I-Type (JALR)	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]

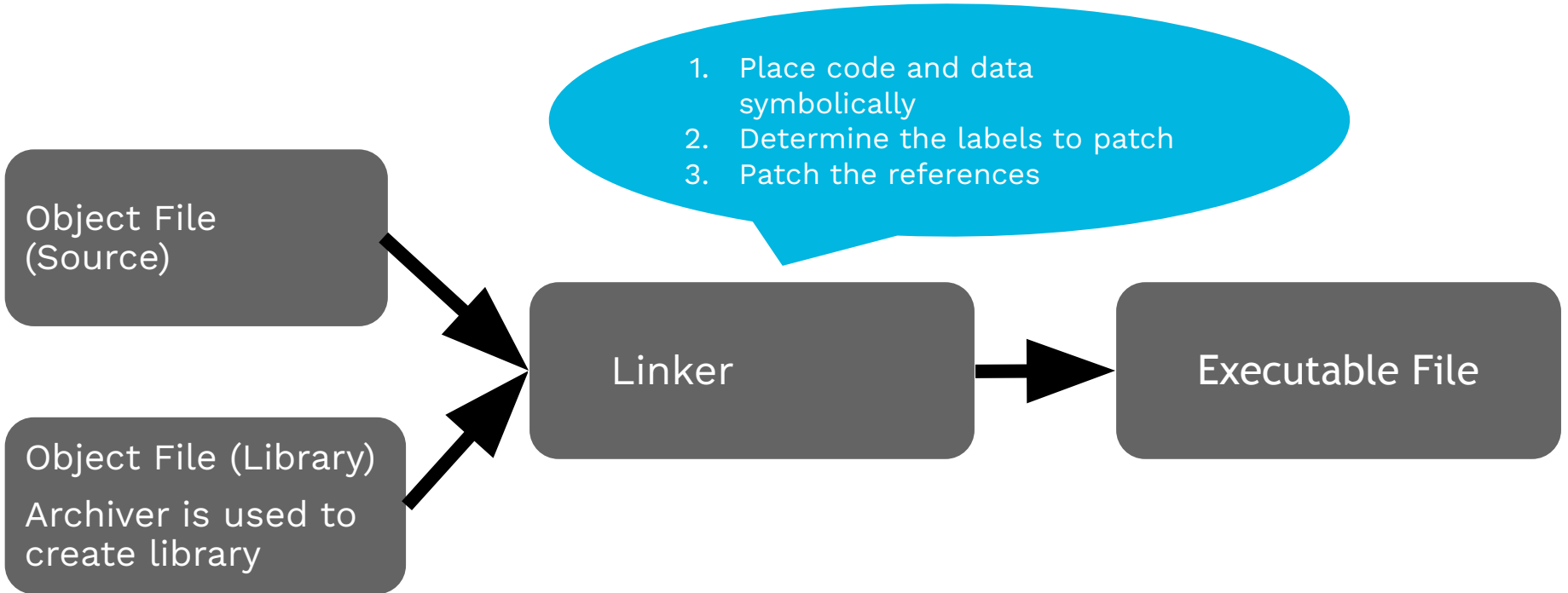
Upper Immediate Type (U-Type)

immediate[31:12]	rd [11-7]	opcode [6-0]
U-immediate[31-12]	dest	LUI/AUIPC

- LUI places the U-immediate value in the top 20 bits of the destination register rd, filling in the lowest 12 bits with zeros.
- With LUI, $rd = imm \ll 12$
- AUIPC forms a 32-bit offset from the 20-bit U-immediate, filling in the lowest 12 bits with zeros, adds this offset to the pc, then places the result in register rd
- With AUIPC, $rd = pc + (imm \ll 12)$

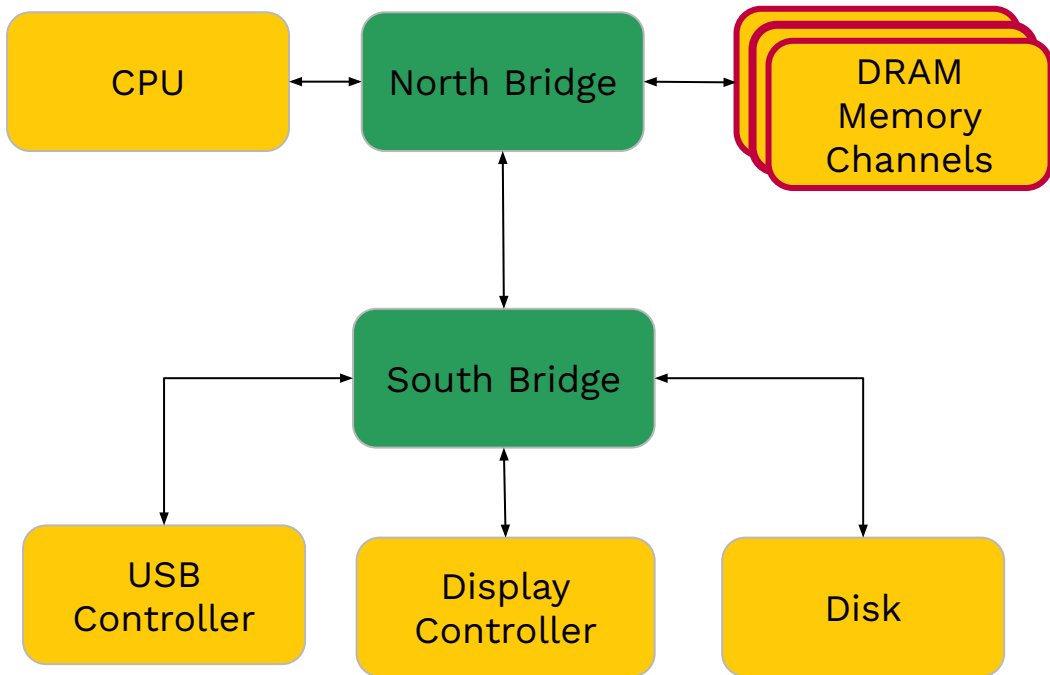
R-Type	func7 [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
I-Type (Shift)	func7 [31-25]	Immediate [24-20]	rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
S-Type	immediate [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	immediate [11-7]	opcode [6-0]
B-Type	immediate [31-25]	rs2 [24-20]	rs1 [19-15]	func3 [14-12]	Immediate [11-7]	opcode [6-0]
J-Type (JAL)	immediate [31:12]				rd [11-7]	opcode [6-0]
I-Type (JALR)	immediate [31-20]		rs1 [19-15]	func3 [14-12]	rd [11-7]	opcode [6-0]
U-Type	immediate [31:12]				rd [11-7]	opcode [6-0]

Linker



The Runtime Inception: Algorithms to Execute Algorithms

What is Physical Memory?



What is physical memory?

- *Stores the data on which the hardware can be accessed without system software.*
- *Otherwise called main memory.*
- *Actual addresses where instructions/data bytes of process are stored.*
- *In early days, this is where all the memory were allocated when a program was created.*

How is it Physically Connected?

Northbridge in the SoC fabric

Where is data allocated?

DRAMs (DDR4, DDR5, HBM). Physically placed into DIMMs (Dual Inline Memory Module)

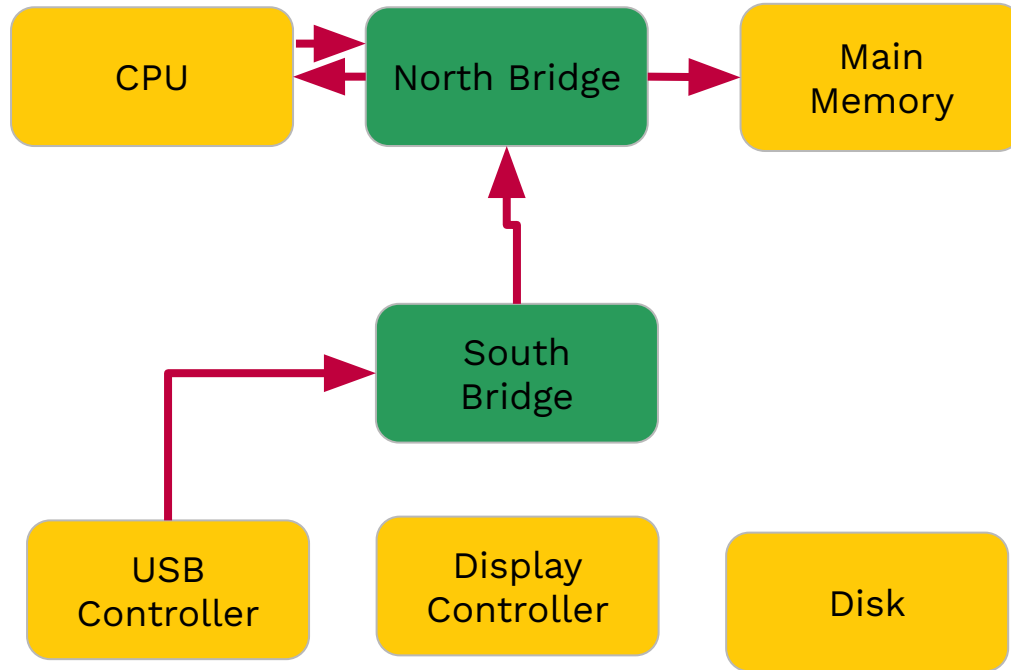
How is it Logically Connected?

Organized as DRAM channels

Who manages data?

- *Loaded by OS using hardware*
- *Accessed by application using hardware*

Runtime: Reading Command



OS is already running a shell program or a GUI

Shell receives a keyboard or mouse interrupts and transfers command into register

load name of the program ("hello") into the memory

Finalizing the interrupt (Enter with Keyboard and click with mouse) to call loader.

The Runtime: Steps of a Loader

Reads header: Size and location

Where to find this?

Creates address space in memory

Which Memory?

Writes the instructions and data into memory

Loader does this!

Loads stack for arguments of the main func

What is stack?

Creates the context of the process and determines the segments of the process

Again, What is context and segment?

Adds the process to the scheduling queue

Waits for Process Scheduler

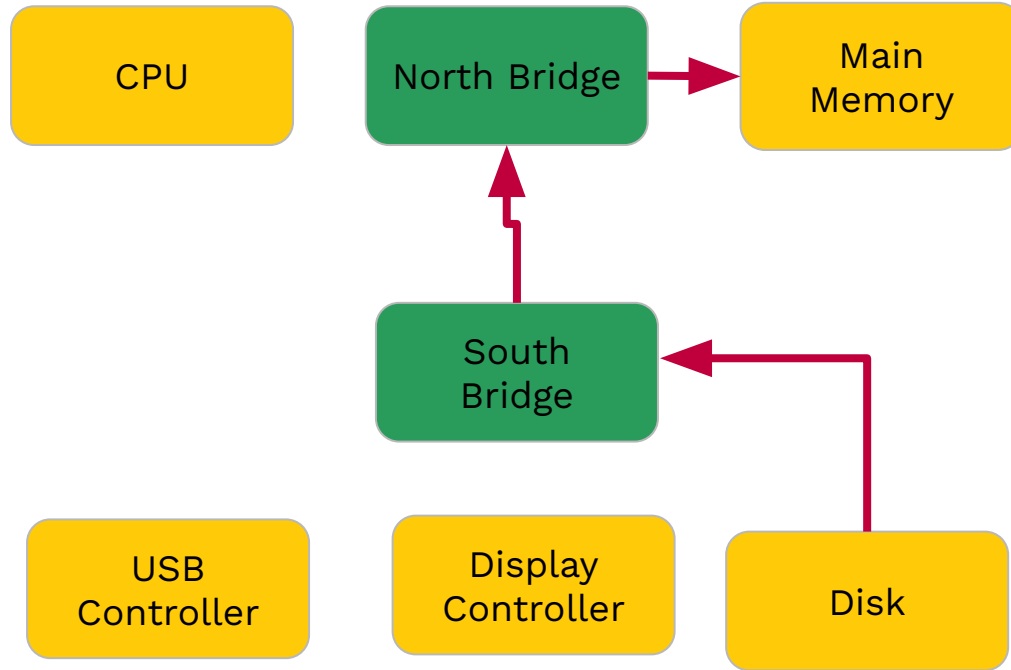
Initializes the control and status registers

Which ones?

Call the program's main function

Thank God!

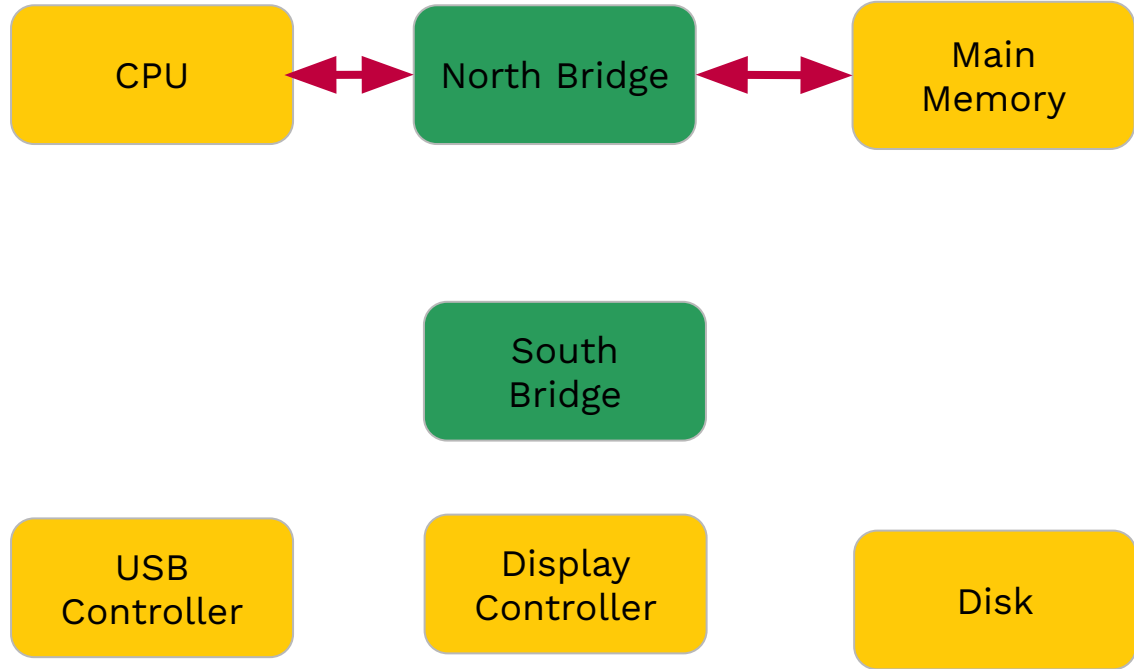
Runtime: Loading an Executable



Loader is called to transfer the binary from disk to main memory

Done using Direct Memory Access (DMA) Hardware. DMA does not use CPU or system bus

The Runtime: Application Execution

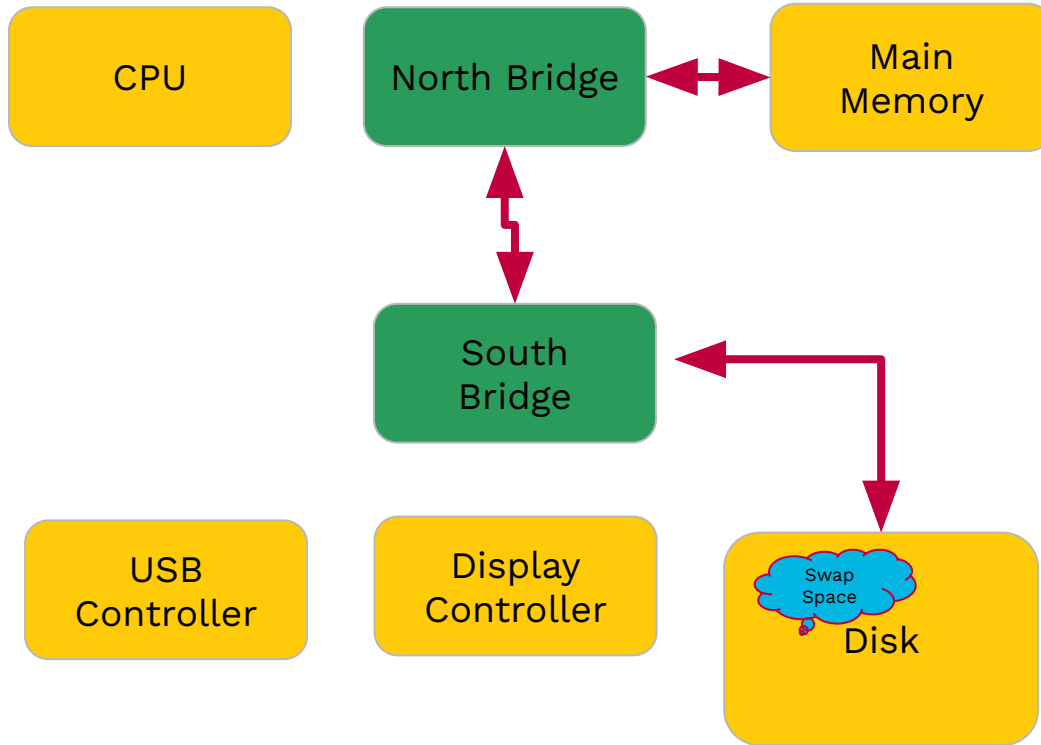


We have to design the hardware to understand the binary and execute it to produce output.

Execution happens mostly on CPU by pulling data from main memory.

Why
Mostly
?

What is Virtual Memory?



What is virtual memory?

Stores the data on which the application operates.

Where is virtual memory allocated?

Swap Space or Disk

Who manages virtual memory?

The memory management system of the Operating System

Why is virtual memory required?

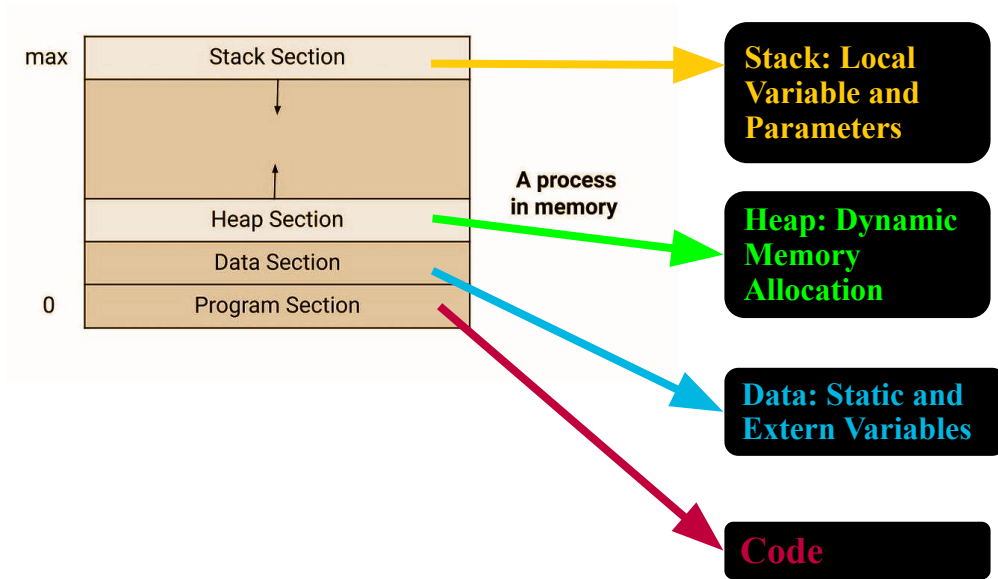
- ☐ One/more program needs memory more than the physical memory size.
- ☐ Protection of one application from the other.
- ☐ Virtual Memory space is unique to a process.
- ☐ Just to relocate the code

How is it Implemented?

Processor generates virtual addresses but works on physical address. The translation of this VA to PA is done using:

- ☐ Segmentation
- ☐ Paging

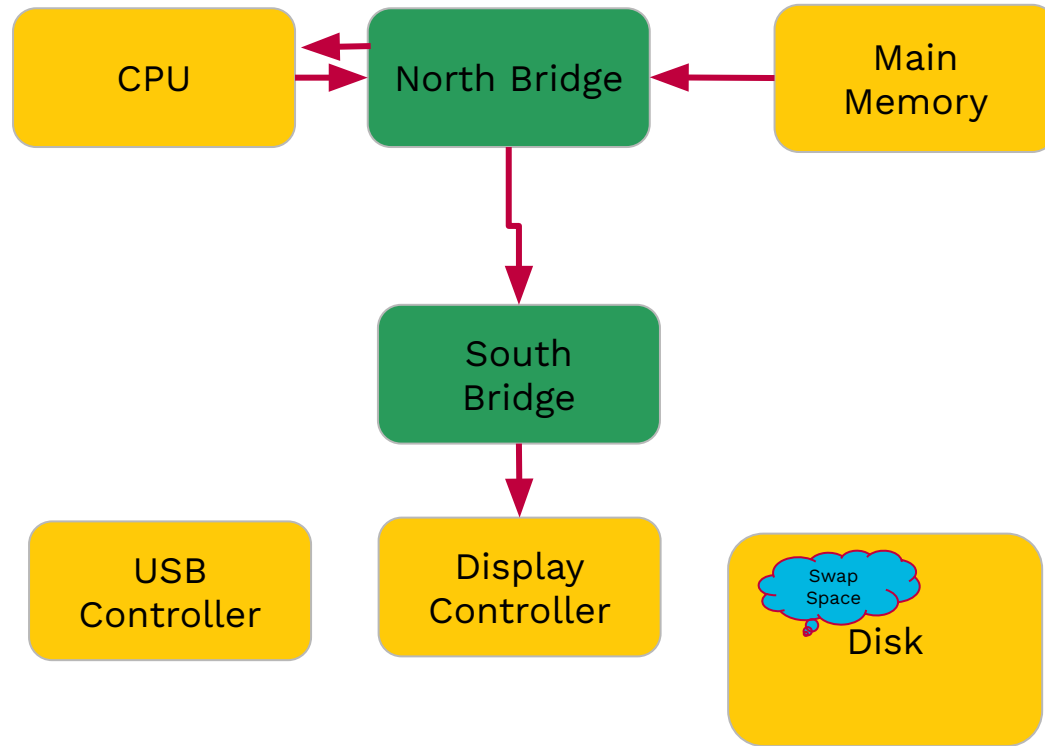
Memory Segments



```
static int i, j;  
extern int k;
```

```
int add(int a, int b){  
    int c = a + b;  
    int *p = (int*) malloc(20);  
    return c;  
}
```

The Runtime: Writing Output

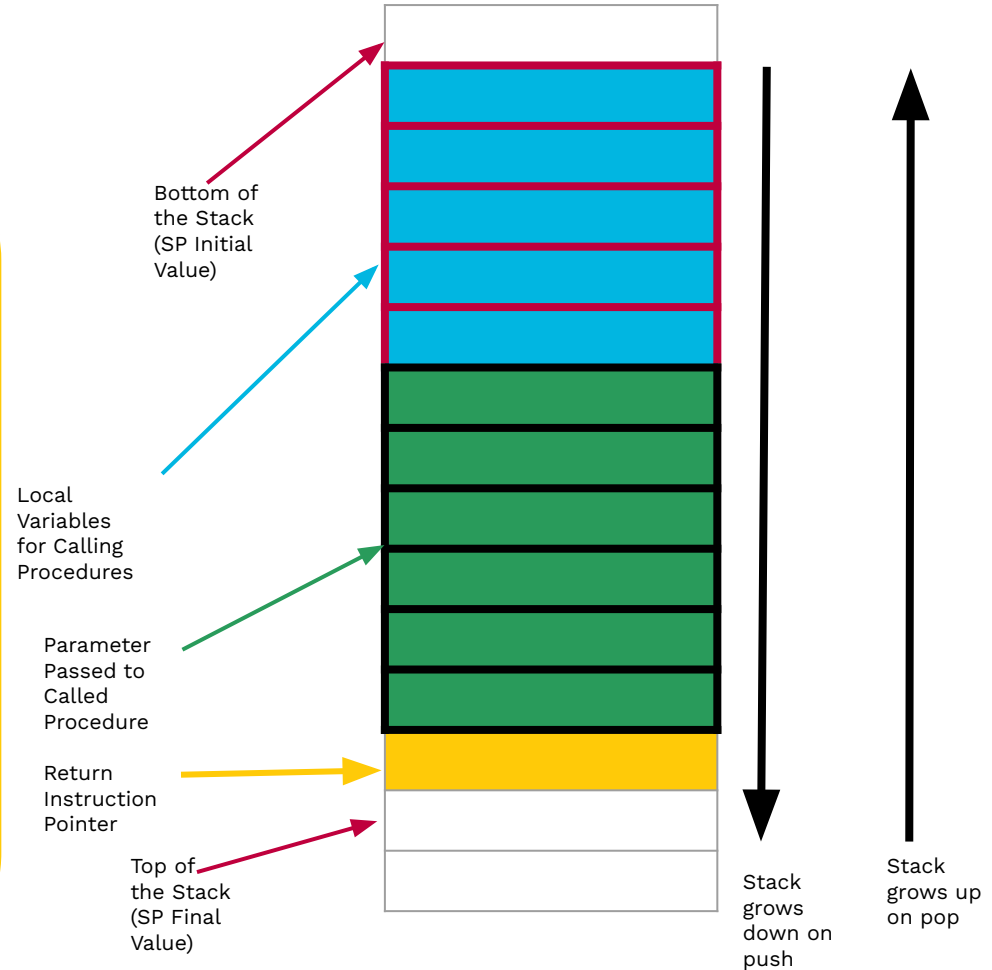


Data is copied from memory to the CPU via North Bridge

The data is then written from registers to the display controller registers via north and south bridge.

Stacks: Data Linked Information

- A stack can be upto 4 GB - max segment size
- This is specific section in the main memory.
- Stack grows down the memory when items are pushed into the stack
- Stack shrinks up the memory when items are popped from the stack
- Each task can be given own stack
- The current stack is only available for the process currently executing.
- Each frame contains local variable, parameters passed to another procedure, other information regarding procedure
- **Stack frame base pointer (SP)** provides easy access to data structures passed onto the stack
- **Return Instruction pointer** : CALL pushes the address in the PC register onto the current stack
- It points to the instruction where execution of the calling procedure should resume following return
- **Near call** : CALL inside same code segment - typically local procedure calls
- **Far call**: CALL to a different code segment - typically OS function calls or some library calls



*“Good Programmers understand what
is the machine, they are Programming!”*

*“Good Architects understand for whom
they are Building!”*

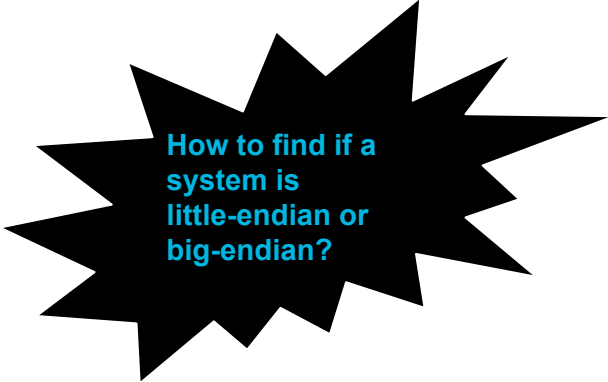
Which one are you?

Great! We are ready with
Executable and Loaded it.
But, how it runs?

Interpreting Memory Address

Little Endian

- Last byte of binary representation of multi-byte data-type is stored first
- 0x01234567 is stored in the memory as 67 45 23 01
- It's easy to cast the value to a smaller type like from `int16_t` to `int8_t` since `int8_t` is the byte at the beginning of `int16_t`.



How to find if a system is little-endian or big-endian?

Big Endian

- First byte of binary representation of multi-byte data-type is stored first
- 0x01234567 is stored in the memory as 01 23 45 57
- More human understandable
- Effective for signed numbers because 1st bit is signed bit.